

# RAG + Agent Eval Demo (Nimbus Cloud Storage support bot)

---

A small, fully-runnable demo of the thing you asked for:

Instrument RAG + Agent, trace every LLM/tool call + latency + cost, build a 50-question eval set, score Precision@k / Faithfulness / Answer Relevancy, and produce a cost dashboard.

It runs **completely offline, no API key needed** — the "LLM" is a mock ( `src/llm.py` ) that answers extractively from retrieved context, so you can see the whole pipeline shape and swap in a real model later with a small, clearly-marked change.

## Project layout

---

|                                     |                                                                                                        |
|-------------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>rag_agent_demo/</code>        |                                                                                                        |
| ├─ <code>corpus/</code>             | 10 fake product-doc .txt files (the knowledge base)                                                    |
| ├─ <code>src/</code>                |                                                                                                        |
| ├─ <code>retriever.py</code>        | TF-IDF retrieval ("R" in RAG)                                                                          |
| ├─ <code>llm.py</code>              | Mock LLM + reference code for a real Anthropic call                                                    |
| ├─ <code>tools.py</code>            | One example tool (calculator) the agent can call                                                       |
| ├─ <code>agent.py</code>            | Orchestrator: routes question -> tool or RAG, traces everything                                        |
| └─ <code>tracer.py</code>           | Records every span (retrieval/llm/tool) -> JSON trace files                                            |
| ├─ <code>evals/</code>              |                                                                                                        |
| ├─ <code>eval_questions.json</code> | 50 questions with ground-truth doc + expected keywords                                                 |
| ├─ <code>metrics.py</code>          | Precision@k, Faithfulness, Answer Relevancy (manual, + RAGAS)                                          |
| ├─ <code>run_eval.py</code>         | Runs all 50 questions, writes results + traces                                                         |
| ├─ <code>cost_dashboard.py</code>   | Aggregates cost/tokens, writes report + chart                                                          |
| ├─ <code>traces/</code>             | <- one JSON trace file per query (generated)                                                           |
| └─ <code>results/</code>            | <- <code>eval_results.csv</code> , <code>eval_summary.md</code> , <code>cost_report.md</code> , chart. |
| └─ <code>run_demo.py</code>         | Interactive CLI to try single questions                                                                |

## How to run it

---

```
pip install -r requirements.txt
```

```
# 1. Try it interactively
python run_demo.py
```

```
# 2. Run the full 50-question eval (writes evals/results/*, evals/traces/*)
python -m evals.run_eval
```

```
# 3. Build the cost dashboard from the eval results
python -m evals.cost_dashboard
```

Open `evals/traces/<trace_id>.json` after any run — that's the exact shape of data Langfuse/LangSmith store per request (see below).

# How each piece maps to what you asked for

---

## 1. Tracing every LLM call / tool call / latency / cost

---

`src/tracer.py` implements a minimal `Trace` + `Span` model:

- `Span` = one step (retrieval, `llm_call`, or `tool_call`), records input/output, `latency_ms`, `prompt_tokens`, `completion_tokens`, `cost_usd`.
- `Trace` = one full user query, holds a list of spans, plus totals.
- Every trace is written to `evals/traces/<trace_id>.json`.

To swap this for the **real Langfuse or LangSmith SDK**, see the comment block at the bottom of `tracer.py` — it's ~10 lines: Langfuse gives you `trace.span(...)` / `trace.generation(...)` objects that push straight to their hosted dashboard instead of a local JSON file; LangSmith uses `@traceable` decorators. The data you're capturing (input, output, latency, tokens, cost) is identical — this demo just writes it to disk instead of to a hosted UI, so it works with zero signup/API key.

## 2. Eval set of 50 questions + metrics

---

`evals/eval_questions.json` — 50 questions (5 per doc x 10 docs), each with a `ground_truth_doc_id` (for retrieval scoring) and `expected_keywords` (for a sanity-check keyword-recall score).

`evals/metrics.py` implements, manually (no LLM judge needed):

- **Precision@k** — of the top-k retrieved docs, how many are the actual right one? Needs ground truth, so it's exact, not a heuristic.
- **Faithfulness** — splits the answer into sentences and checks how many are grounded (TF-IDF similarity) in the retrieved context — catches hallucination.
- **Answer Relevancy** — TF-IDF similarity between question and answer — catches "answered the wrong thing" / rambling.
- (bonus) **Keyword Recall** — did the answer mention the specific facts we know are correct.

A comment block at the bottom of `metrics.py` shows the ~5-line swap to use **real RAGAS** instead (it uses an LLM-as-judge, which is more accurate but costs an API call per metric per row).

`evals/run_eval.py` runs all 50, scores each, and writes `evals/results/eval_results.csv` (one row per question) and `evals/results/eval_summary.md` (averaged scores + a list of the worst-performing questions to go debug).

**A real finding from this demo's first run:** average Precision@3 came out to ~0.33. That's not a bug — with only 1 correct doc per question and k=3, the mathematical ceiling for Precision@3 is 1/3 whenever retrieval gets the right doc at rank 1. That's exactly the kind of thing an eval set is supposed to surface: without it you'd never notice your metric's ceiling didn't match your data.

### 3. Cost dashboard

---

`evals/cost_dashboard.py` reads the CSV from step 2 and produces:

- `evals/results/cost_report.md` — total/avg tokens, total/avg cost, and the number you actually care about: **projected \$ per 1,000 queries**.
- `evals/results/cost_by_query.png` — bar chart of cost per query, so you can spot outlier-expensive questions at a glance.

Pricing is a constant in `src/llm.py` (`PRICE_PER_1M_INPUT_TOKENS / PRICE_PER_1M_OUTPUT_TOKENS`) — swap in your real provider's current per-model rate.

## Going from mock to real

---

1. Set `ANTHROPIC_API_KEY` and `USE_REAL_LLM=1` — `src/llm.py` already has a working `_call_real_llm()` that calls the real Anthropic API and pulls real token counts from `resp.usage`.
2. Swap `src/retriever.py`'s TF-IDF for real embeddings + a vector DB (Chroma/Pinecone/pgvector) — the `retrieve(query, k)` interface stays the same, so nothing else in the pipeline needs to change.
3. Swap the tracer for real Langfuse/LangSmith (see comment block in `tracer.py`) to get a hosted dashboard instead of local JSON files.
4. Swap the manual metrics for real RAGAS (see comment block in `metrics.py`) once you're ready to spend tokens on LLM-judged scores.

Each of these is an isolated, one-file change — that's the whole point of building the demo with these seams in place.